SIMATS ENGINEERING
ASSESSMENT TOOL 1: Design Task

COURSE NAME: ARTIFICIAL INTELLIGENCE **N.sri vara Deepak**

COURSE CODE: CSA17 **192572075**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.*(BTL 5)

Assessment Tool Weightage: 50%

Code of Conduct:

I N.sri vara Deepak(192572075)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Deepak.N***

Total Marks: 100

Each question carries equal marks.

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyse real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

1. End-to-End System Design

Problem

Design a scalable AI system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations.

Solution

1. System Architecture

Social Media Sources
↓
Data Ingestion
↓
Kafka / Streaming Layer
↓
PySpark
↓
RDD Processing
↓
Cleaning → Feature Extraction → Sentiment Analysis
↓
AI Model
↓
Recommendation Engine
↓
Intelligent Agent
↓
User Recommendations

2. Main Components

1. **Data Source**
   1.1. Collects posts, comments, hashtags, and user interactions.
   1.2. Data can arrive continuously from social media platforms.
2. **Data Ingestion**
   2.1. Kafka or another streaming system receives incoming data.
   2.2. Provides continuous data to the processing layer.
3. **PySpark Processing**
   3.1. PySpark distributes processing across multiple worker nodes.
   3.2. RDDs provide fault-tolerant distributed data processing.
4. **RDD Operations**
   4.1. map() – extracts required fields.
   4.2. filter() – removes irrelevant data.
   4.3. flatMap() – extracts hashtags or keywords.
   4.4. reduceByKey() – calculates keyword frequency.
   4.5. groupByKey() – groups information by users/topics.

3. Example RDD Operations:

```
posts = sc.textFile("social_media.txt")

cleaned = posts.map(lambda x: x.lower())

filtered = cleaned.filter(lambda x: len(x) > 10)

hashtags = filtered.flatMap(
    lambda x: [word for word in x.split() if word.startswith("#")]
)

frequency = hashtags.map(
    lambda x: (x, 1)
).reduceByKey(lambda a, b: a + b)

print(frequency.collect())
```

4. Intelligent Agent Integration

1. Agent
2. ↓
3. Perceives social-media trends
4. ↓
5. Analyzes user preferences
6. ↓
7. Makes recommendation
8. ↓
9. Provides personalized content

The agent can recommend:

- Trending topics

- Products

5. Data Flow

Social Media

↓

Data Collection

↓

Distributed RDD Processing

↓

Feature Extraction

↓

AI Prediction

↓

Agent Decision

↓

Recommendation

6. Advantages

- Handles large volumes of data.
- Distributed processing improves performance.
- RDDs provide fault tolerance.
- Recommendations can be personalized.
- System can scale by adding worker nodes.

Conclusion

The proposed system combines **real-time data processing, PySpark RDDs, AI models, and intelligent agents** to provide scalable and personalized social-media recommendations.

2. Distributed Intelligent Agent System

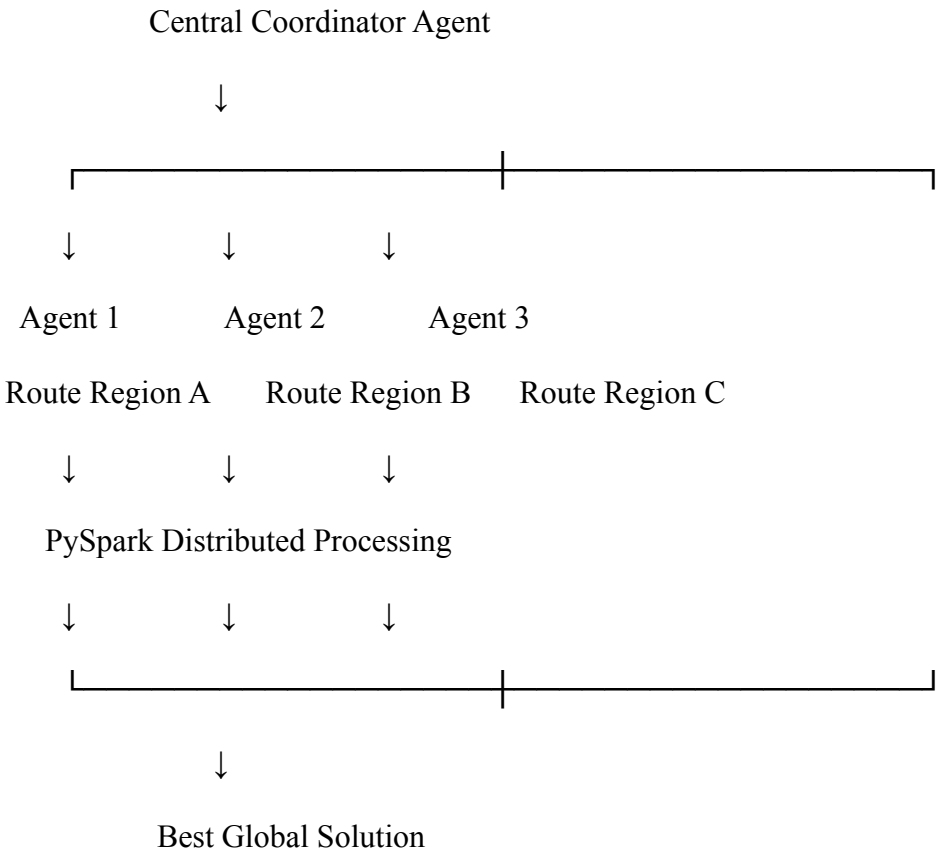
Problem

Develop a multi-agent architecture using PySpark for distributed processing and agent collaboration to solve a large-scale optimization problem.

Solution

A suitable application is **large-scale delivery route optimization**.

1. Architecture



2. Agents

Agent 1 – Data Agent

- Collects delivery locations.
- Processes customer and vehicle information.

Agent 2 – Route Agent

- Generates possible routes.
- Calculates route distances.

Agent 3 – Optimization Agent

- Searches for efficient routes.
- Minimizes distance and delivery time.

Agent 4 – Coordinator Agent

- Receives results from other agents.
- Compares solutions.
- Selects the best global solution.

3. PySpark RDD Processing

```
routes = sc.parallelize([

    ("R1", 120),

    ("R2", 95),

    ("R3", 140),

    ("R4", 80),

    ("R5", 110)

])



best_route = routes.reduce(

    lambda x, y: x if x[1] < y[1] else y

)



print("Best Route:", best_route)
```

Output:

Best Route: ('R4', 80)

4. Agent Collaboration

Agent 1 → Processes customer data

↓

Agent 2 → Generates routes

↓

Agent 3 → Optimizes routes

↓

Agent 4 → Selects global solution

↓

Final Decision

Agents communicate their partial results to the coordinator.

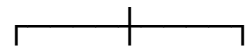
5. Scalability

PySpark divides the optimization problem into smaller tasks:

Large Dataset

↓

Partition

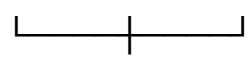


↓ ↓ ↓

P1 P2 P3

↓ ↓ ↓

W1 W2 W3



↓

Combine Results

6. Advantages

- Parallel processing.
- Multiple agents can work simultaneously.
- Faster optimization.
- Handles large datasets.
- Fault-tolerant RDD processing.
- Easy horizontal scaling.

Conclusion

A **multi-agent PySpark architecture** allows different agents to independently solve portions of a large optimization problem and collaborate to produce a global solution efficiently.

3. Big Data AI Pipeline Design

Problem

Design a distributed AI pipeline using RDD transformations for preprocessing, feature engineering, and model training.

Solution

1. Pipeline Architecture

Raw Big Data
↓
Data Ingestion
↓
RDD Creation
↓
Data Cleaning
↓
Data Transformation
↓
Feature Engineering
↓
Training / Testing Split
↓
Distributed Model Training
↓
Evaluation
↓
Prediction

2. Data Preprocessing

Suppose the dataset contains:

```
ID,Age,Income,Experience,Class
1,25,30000,2,A
2,30,45000,5,B
3,22,25000,1,A
```

Create an RDD:

```
data = sc.textFile("data.csv")

header = data.first()

rows = data.filter(lambda x: x != header)
```

3. Cleaning

```
cleaned = rows.map(
   lambda x: x.split(",")
).filter(
   lambda x: len(x) == 5
)
```

4. Feature Engineering

```
features = cleaned.map(
   lambda x: (
      [float(x[1]), float(x[2]), float(x[3])],
      x[4]
   )
)
```

The features are:

Age
Income
Experience

5. RDD Transformations

Operation	Purpose
map()	Transform each record
filter()	Remove invalid records
flatMap()	Generate multiple values
reduceByKey()	Aggregate values
groupByKey()	Group records
sortByKey()	Sort distributed data

6. Training and Testing

```
train, test = features.randomSplit([0.8, 0.2])
```

Training data is used to build the AI model, while testing data evaluates its performance.

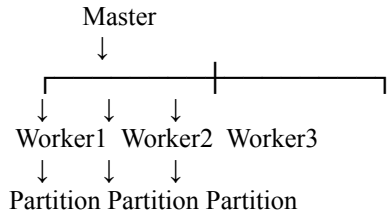
7. Fault Tolerance

RDDs maintain **lineage information**.

Input RDD
↓
map()
↓
filter()
↓
feature RDD

If a partition is lost, Spark can recompute it from the lineage instead of processing the entire dataset again.

8. Scalability



More worker nodes can be added when data volume increases.

Conclusion

The RDD-based AI pipeline provides **distributed preprocessing, feature engineering, model training, scalability, and fault tolerance**, making it suitable for large-scale AI applications.

4. Real-Time Decision-Making System

Problem

Construct an AI system using RDD-based streaming processing and intelligent agents for real-time decision-making in smart traffic or healthcare.

Example: Smart Traffic Management System

1. Architecture

Traffic Sensors / Cameras

↓

Data Streaming

↓

PySpark

↓

Streaming RDDs

↓

Data Cleaning + Analysis

↓

Traffic Prediction Model

↓

Intelligent Traffic Agent

↓

Decision Making

↓

Traffic Signal Control

2. Data Collection

Traffic sensors provide:

- Vehicle count
- Average speed
- Road occupancy
- Traffic density
- Accident information

3. RDD Processing

```
traffic = sc.textFile("traffic_data.txt")


records = traffic.map(
    lambda x: x.split(",")
)


valid = records.filter(
    lambda x: len(x) >= 4
)


density = valid.map(
```

```
    lambda x: (x[0], int(x[1]))
)

total = density.reduceByKey(
    lambda a, b: a + b
)
```

4. Intelligent Traffic Agent

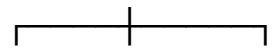
The agent receives traffic information and decides:

Traffic Density

↓

Agent

↓



↓ ↓ ↓

Low Medium High

↓ ↓ ↓

Normal Adjust Increase

Signal Signal Green Time

Example:

If traffic density > threshold

↓

Increase green-light duration

↓

Reduce congestion

5. Real-Time Decision Process

Sensor Data

↓

Streaming

↓

RDD Processing

↓

Traffic Analysis

↓

AI Prediction

↓

Agent Decision

↓

Traffic Signal Update

6. Benefits

- Real-time monitoring.
- Faster decision-making.
- Reduced traffic congestion.
- Distributed processing.
- Scalable architecture.
- Automated traffic control.

Conclusion

The combination of **PySpark streaming, RDD processing, AI prediction, and intelligent agents** enables real-time and autonomous traffic management.

5. Cloud-Based Distributed AI Application

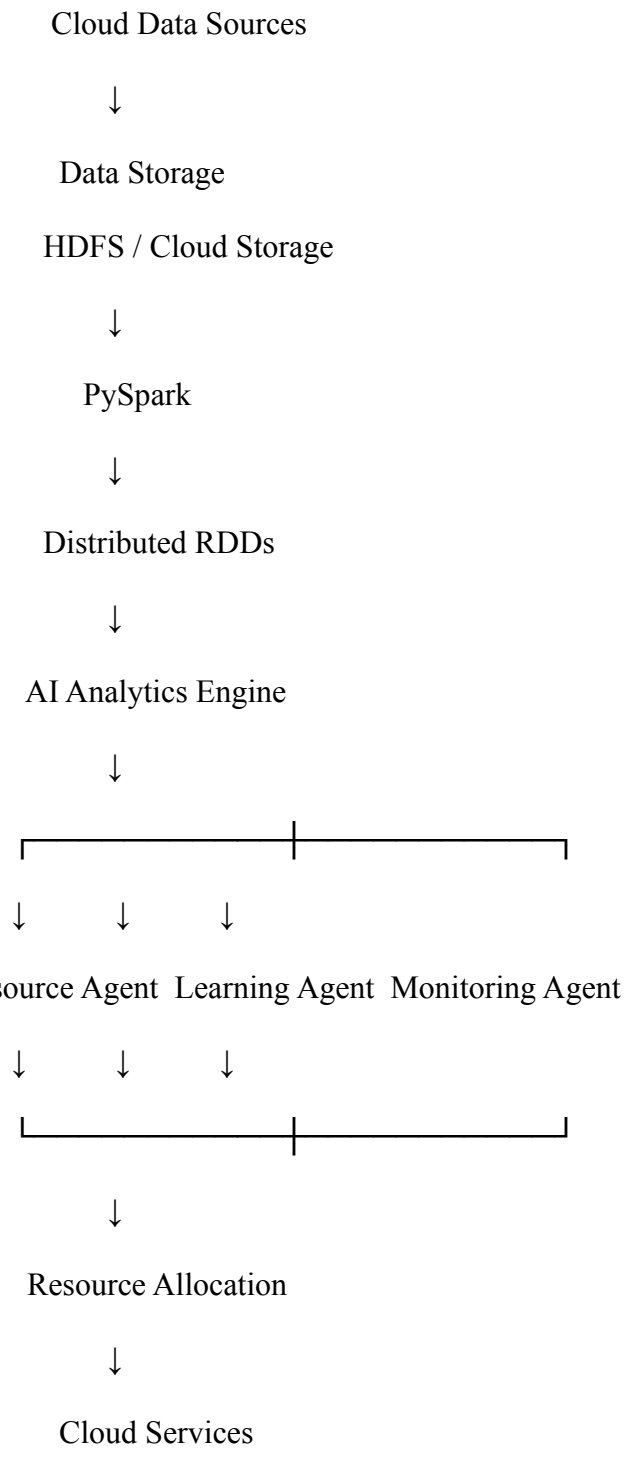
Problem

Design a cloud-enabled AI system using PySpark and intelligent agents for large-scale analytics, dynamic resource allocation, and adaptive learning.

Solution

A suitable application is a **cloud-based intelligent resource management system**.

1. Architecture



2. Resource Agent

Monitors:

- CPU usage
- Memory usage
- Network usage
- Number of active tasks

It dynamically allocates resources according to workload.

3. Learning Agent

- Monitors historical workload.
- Learns workload patterns.
- Predicts future resource requirements.
- Improves allocation decisions.

4. PySpark RDD Processing

```
usage = sc.textFile("resource_usage.csv")



records = usage.map(

    lambda x: x.split(",")

)



cpu_data = records.map(

    lambda x: (x[0], float(x[1]))

)



average_cpu = cpu_data.map(

    lambda x: x[1]

).mean()



print("Average CPU:", average_cpu)
```

5. Dynamic Resource Allocation

Low Workload

↓

Reduce Resources

↓

Save Cost

High Workload

↓

Increase Resources

↓

Improve Performance

6. Adaptive Learning

Historical Data

↓

PySpark Processing

↓

AI Model

↓

Workload Prediction

↓

Learning Agent

↓

Resource Allocation

↓

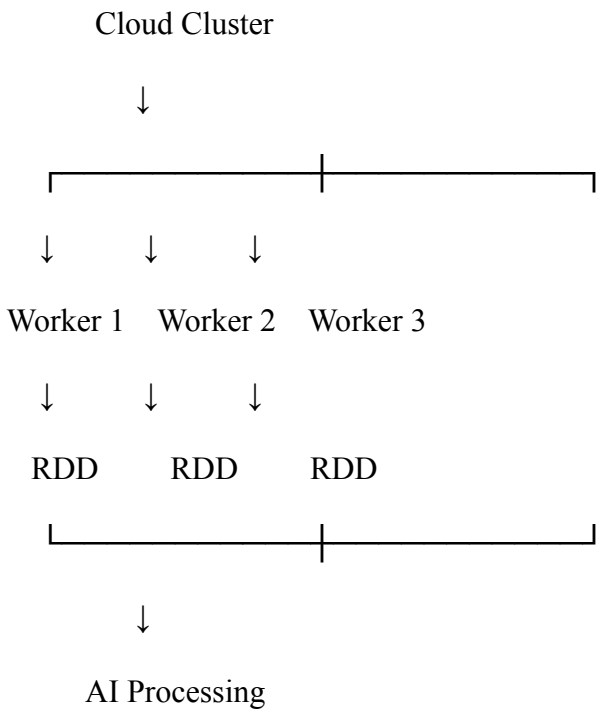
New Performance Data

↓

Model Update

This creates a feedback loop in which the system continuously improves its decisions.

7. Cloud Scalability



Additional workers can be added when the workload increases.

8. Advantages

- Large-scale data processing.
- Elastic cloud resources.
- Dynamic resource allocation.
- Fault tolerance through Spark.
- Adaptive AI learning.
- Reduced infrastructure cost.
- High availability and scalability.

Conclusion

A cloud-based distributed AI system combining **PySpark, RDDs, intelligent agents, and adaptive learning** can automatically analyze large datasets, predict workload requirements, and dynamically allocate cloud resources.

RUBRICS FOR CALCULATION:

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, wellstructured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/ actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative,	Some innovation	Basic idea	No originality

	realistic, and impactful solution			
Clarity, Presentation & Documentation	Very clear, neat diagrams, wellorganized report	Mostly clear	Somewhat organized	Poor presentation